SISTEMAS DISTRIBUÍDOS

Prof Priscila Marques Kai

FLASK

Flask é um framework leve para aplicações web WSGI (*Web Server Gateway Interface*).

- Foi projetado para facilitar e agilizar o início de aplicações, com a capacidade de escalar para aplicações complexas.

FLASK

Exemplo simples:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "<p>Bem vindo ao SD Acadêmico!</p>"

if __name__ == "__main__":
    app.run()
```

Para visualizar use `http://127.0.0.1:5000/`

PRÁTICA COM PYTHON - UTILIZANDO FLASK PARA CRIAR UM SISTEMA DE MATRÍCULA ONLINE SIMPLES

Itens a abordados abordados no exemplo:

- cliente-servidor;
- requisição e resposta;
- vagas limitadas;
- duplicidade de requisição;

PRÁTICA COM PYTHON - UTILIZANDO FLASK PARA CRIAR UM SISTEMA DE MATRÍCULA ONLINE SIMPLES

O que a aplicação deverá conter:

- Verificar se o servidor está funcionando
- uma lista de disciplinas com número de vagas;
- alunos enviando pedido de matrícula;
- o servidor verificando:
    - se a disciplina existe;
    - se ainda há vaga;
    - se o aluno já está matriculado;

PRÁTICA COM PYTHON – UTILIZANDO FLASK PARA CRIAR UM SISTEMA DE MATRÍCULA ONLINE SIMPLES

O que a aplicação deverá conter:

- Verificar se o servidor está funcionando
- uma lista de disciplinas com número de vagas;
- alunos enviando pedido de matrícula;
- o servidor verificando:
    - se a disciplina existe;
    - se ainda há vaga;
    - se o aluno já está matriculado;

PRÁTICA COM PYTHON

- Crie utilizando Flask, uma rota para verificar se o servidor está operando
  - Use:

```
from flask import Flask, jsonify
```

  - Para adicionar uma rota:
    - methods={"GET"} → para recuperar dados
    - methods={"POST"} → para o envio de dados para criar ou atualizar recursos.
      Exemplo:

```
@app.route("/rota", methods={"GET"})
```

PRÁTICA COM PYTHON – UTILIZANDO FLASK PARA CRIAR UM SISTEMA DE MATRÍCULA ONLINE SIMPLES

O que a aplicação deverá conter:

- Verificar se o servidor está funcionando
- uma lista de disciplinas com número de vagas;
- alunos enviando pedido de matrícula;
- o servidor verificando:
  - se a disciplina existe;
  - se ainda há vaga;
  - se o aluno já está matriculado;

PRÁTICA COM PYTHON

- Crie utilizando Flask, uma rota que forneça uma lista de disciplinas com número de vagas para o usuário

```
@app.route("/disciplinas", methods=["GET"])
```

- Crie uma variável do tipo dicionário para guardar as informações das disciplinas

  Exemplo:

```
disciplinas = {
    "EC0000A01":{"nome":"Algoritmos e Estruturas de Dados I" ,"Vagas":40},
    "EC0000A02":{"nome":"Algoritmos e Estruturas de Dados II" ,"Vagas":40}
}
```

PRÁTICA COM PYTHON – UTILIZANDO FLASK PARA CRIAR UM SISTEMA DE MATRÍCULA ONLINE SIMPLES

O que a aplicação deverá conter:

- Verificar se o servidor está funcionando
- uma lista de disciplinas com número de vagas;
- alunos enviando pedido de matrícula;
- o servidor verificando:
  - se a disciplina existe;
  - se ainda há vaga;
  - se o aluno já está matriculado;

PRÁTICA COM PYTHON

Abra o arquivo app_matricula.py e complemente o código.

Para executar o cliente utilize o arquivo app_cliente.py

PRÁTICA COM PYTHON - UTILIZANDO FLASK PARA CRIAR UM SISTEMA DE MATRÍCULA ONLINE SIMPLES

Itens a abordados abordados no exemplo:

- cliente-servidor;
- requisição e resposta;
- vagas limitadas;
- duplicidade de requisição;

SISTEMAS DISTRIBUÍDOS

Uma implementação simples em Python do modelo **Cliente-Servidor com sockets**

Considere como exemplo dois programas que precisam trocar dados em um modelo cliente-servidor

- Podem estar no mesmo computador (localhost) ou em máquinas diferentes (rede local ou internet).
- Analogia da operação: telefone → alguém liga (cliente), alguém atende (servidor)

Cliente e Servidor

Servidor: espera conexões → “porta aberta”.

Cliente: inicia a conversa → tenta entrar na porta.

Sem servidor, não há com quem falar.

IP e Porta

- **IP** identifica a máquina na rede. Exemplo: 127.0.0.1 → localhost (a própria máquina).
- **Porta** identifica o programa dentro da máquina. Exemplo: HTTP → porta 80.
- Cliente precisa saber: IP + Porta.

Introdução

- **O que é um Socket**
  - É o canal de comunicação entre cliente e servidor.
  - Abstração criada pelo sistema operacional.
  - Cada socket é identificado por (IP, porta).
  - Pense como um cano por onde passam mensagens.

Programação de Sockets em Python

Um **socket** é um mecanismo usado para permitir a comunicação entre dois pontos (nós) em uma rede.

Ele funciona como uma “porta de entrada e saída” para troca de mensagens.

- De um lado, um **servidor** cria um socket e fica escutando em um endereço (IP) e porta específicos, aguardando conexões.
- Do outro lado, um **cliente** utiliza outro socket para se conectar ao servidor e, a partir daí, ambos podem trocar informações.

De forma simples, podemos dizer que:

- O **servidor** escuta.
- O **cliente** inicia a conversa.

Programação de Sockets em Python

Para programar com sockets em Python, o primeiro passo é importar a biblioteca **socket** e, em seguida, criar um objeto de socket que será usado para estabelecer a comunicação.

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

parâmetros:

- AF_INET - refere-se à família de endereços IPv4.
- SOCK_STREAM - protocolo TCP orientado à conexão.

Após isso poderemos conectar a um servidor usando o socket.

- só poderemos nos conectar a um servidor se soubermos seu IP.

Fluxo de funcionamento

1. Servidor cria socket e “abre porta”.
2. Cliente cria socket e pede conexão.
3. Cliente envia mensagem → Servidor recebe.
4. Servidor responde → Cliente recebe.
5. Conexão é encerrada.

Programação de Sockets em Python

Podemos descobrir um IP através da instrução:

```
ip = socket.gethostbyname('www.google.com')
imprimir ip
```

parâmetros:

- AF_INET - refere-se à família de endereços IPv4.
- SOCK_STREAM - protocolo TCP orientado à conexão.

Após isso poderemos conectar a um servidor usando o socket.

- só poderemos nos conectar a um servidor se soubermos seu IP.

Programação de Sockets em Python

No lado do servidor, usamos o método bind() para vincular um IP e uma porta a fim de "escutar" por solicitações recebidas naquele IP e porta.

```
ip = socket.gethostbyname('www.google.com')
imprimir ip
```

listen() → modo de escuta

Programação de Sockets em Python

No lado do cliente:

- criamos um objeto socket
- conectamos ao localhost na porta do mesmo modo definido no servidor (a porta na qual o servidor é executado)

```
s.connect(('127.0.0.1', port))
```

- recebemos os dados do servidor

```
s.recv(1024)
```

- fechamos a conexão.

conexao.

```
ip = socket.gethostbyname('www.google.com')
imprimir ip
```

listen() → modo de escuta

Exemplo – Servidor (Python)

```
import socket
s = socket.socket()
s.bind(("127.0.0.1", 5000))
s.listen()
conn, addr = s.accept()
print("Conectado:", addr)
data = conn.recv(1024)
conn.send(str.encode("Ola Cliente"))
conn.close()
```

Exemplo - Cliente (Python)

```
import socket

s = socket.socket()

s.connect(("127.0.0.1", 5000))

s.send(b"Ola Servidor")

data = s.recv(1024)

print("Servidor disse:", data.decode())

s.close()
```

Experimento em sala

Executar o servidor em um terminal.

Executar o cliente em outro terminal.

Observar a troca de mensagens.

Perguntas

O que acontece se mudar a porta só no cliente?

E se mudar 127.0.0.1 para outro IP da rede?

Por que o servidor precisa ficar sempre rodando?

Cliente-Servidor

Cliente-Servidor é a base de sistemas distribuídos.

Socket = meio de comunicação confiável.

Esse modelo será usado em sistemas mais complexos.